Experiment 1: Insertion Sort

Write a program to implement the insertion sort using array.

1. Theory

Insertion sort builds the sorted array one element at a time by dividing it into a sorted and an unsorted part. Each element from the unsorted part is picked and inserted into its correct position within the sorted part, shifting larger elements one step ahead. It works well for small or nearly sorted data sets. Its time complexity is O(n^2) in the worst and average case, and O(n) in the best case. It is a simple, stable, and in-place sorting algorithm.

2. Algorithm

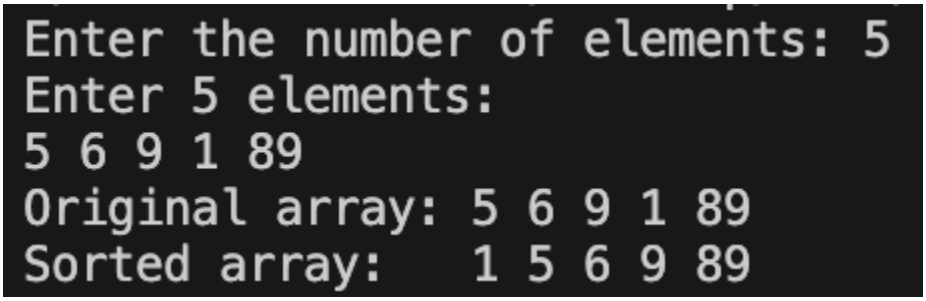
1. Start from the second element (index 1) of the array.
2. Store the current element in a variable key.
3. Compare key with the elements to its left.
4. Shift all elements greater than key one position to the right.
5. Insert key at its correct position.
6. Repeat steps 2-5 for all elements till the end of the array.
7. Stop.

3. Code

```
#include <stdio.h>

// Function to sort array using Insertion Sort
void insertionSort(int arr[], int n)
{
    int i, key, j;
    for (i = 1; i < n; i++)
    {
        key = arr[i]; // element to be inserted
        j = i - 1;
        // Shift elements greater than key one position ahead
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key; // place key at correct position
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
```

```
        printf("%d ", arr[i]);
    printf("\n");
}

int main()
{
    int arr[] = {29, 10, 14, 37, 14, 8};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    insertionSort(arr, n);

    printf("Sorted array:   ");
    printArray(arr, n);

    return 0;
}
```

4. Output



Experiment 2: Selection Sort

Write a program to implement the selection sort using array.

1. Theory

Selection sort repeatedly selects the smallest element from the unsorted part of the array and places it at the beginning of that part. The array is thus divided into a sorted and an unsorted section, with the boundary advancing by one after every pass. It performs a fixed number of comparisons regardless of the initial arrangement of data. Its time complexity is O(n^2) for all cases. It is an easy to implement, in-place sorting algorithm.

2. Algorithm

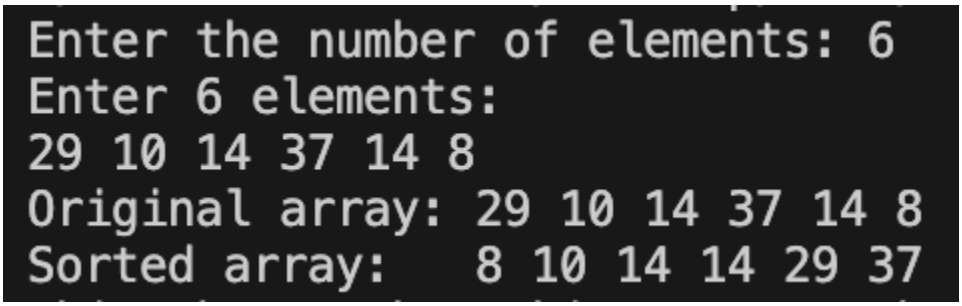
1. Start with the first element as the assumed minimum position.
2. Traverse the unsorted part to find the index of the smallest element.
3. Swap the smallest element with the first element of the unsorted part.
4. Move the boundary of the sorted part one position ahead.
5. Repeat steps 2-4 until only one element remains unsorted.
6. Stop.

3. Code

```
#include <stdio.h>

// Function to sort array using Selection Sort
void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;
    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        // Find index of minimum element in unsorted part
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        // Swap minimum element with first unsorted element
        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```

```
int main() {
    int arr[100], n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    printArray(arr, n);

    selectionSort(arr, n);

    printf("Sorted array:   ");
    printArray(arr, n);

    return 0;
}
```

4. Output



Experiment 3: Linear Search

Write a program to implement the Linear Search using array.

1. Theory

Linear search checks each element of the array one by one, starting from the first, until the required element is found or the array ends. It does not require the array to be sorted, making it suitable for small or unordered lists. Its main drawback is inefficiency on large data sets, since every element may need to be checked. Its time complexity is O(n) in the worst case and O(1) in the best case. It is simple to understand and implement.

2. Algorithm

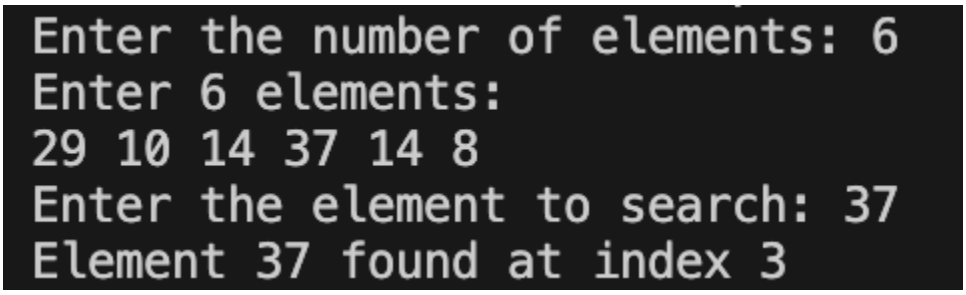
1. Start from the first element of the array.
2. Compare the current element with the key.
3. If it matches, return its index and stop.
4. If not, move to the next element.
5. Repeat steps 2-4 until the key is found or the array ends.
6. If the end is reached without a match, return -1.

3. Code

```
#include <stdio.h>

// Function to search key using Linear Search
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;   // return index if found
    }
    return -1;          // not found
}

int main() {
    int arr[100], n, key;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &key);
```

```
    int result = linearSearch(arr, n, key);

    if (result != -1)
        printf("Element %d found at index %d\n", key, result);
    else
        printf("Element %d not found in array\n", key);

    return 0;
}
```

4. Output



Experiment 4: Binary Search

Write a program to implement the Binary Search using array.

1. Theory

Binary search locates an element in a sorted array by repeatedly dividing the search interval in half. It compares the key with the middle element and discards the half of the array where the key cannot lie, narrowing the search quickly. This makes it much faster than linear search for large sorted data sets. Its time complexity is O(log n). The array must be sorted before binary search can be applied.

2. Algorithm

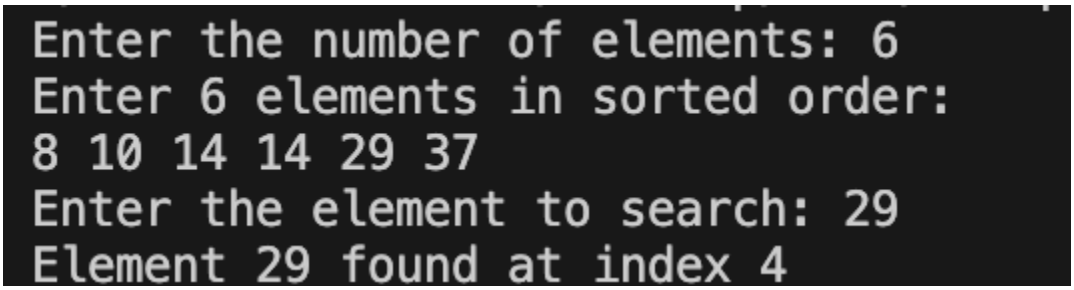
1. Set low to 0 and high to n-1.
2. Repeat while low is less than or equal to high.
3. Find mid = (low + high) / 2.
4. If arr[mid] equals key, return mid and stop.
5. If arr[mid] is less than key, set low = mid + 1.
6. Else set high = mid - 1.
7. If the loop ends without a match, return -1.

3. Code

```
#include <stdio.h>

// Function to search key using Binary Search (array must be sorted)
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;             // key found
        else if (arr[mid] < key)
            low = mid + 1;          // search right half
        else
            high = mid - 1;         // search left half
    }
    return -1;                      // not found
}

int main() {
    int arr[100], n, key;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements in sorted order:\n", n);
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter the element to search: ");
    scanf("%d", &key);

    int result = binarySearch(arr, n, key);

    if (result != -1)
        printf("Element %d found at index %d\n", key, result);
    else
        printf("Element %d not found in array\n", key);

    return 0;
}
```

4. Output



Experiment 5: Merge Sort

Write a program to implement the Merge Sort using array.

1. Theory

Merge sort is a divide-and-conquer algorithm that splits the array into two halves, recursively sorts each half, and then merges the two sorted halves into a single sorted array. It guarantees O(n log n) time complexity in the best, average, and worst case, making it reliable for large data sets. It is a stable sorting algorithm, though it needs extra space proportional to the array size for the merge step.

2. Algorithm

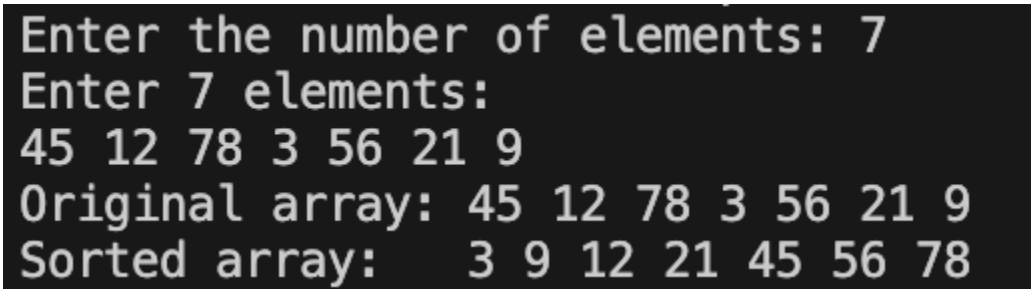
1. If the array has more than one element, find the middle index.
2. Recursively divide and sort the left half.
3. Recursively divide and sort the right half.
4. Merge the two sorted halves into a single sorted array.
5. Repeat the process until the entire array is sorted.
6. Stop.

3. Code

```
#include <stdio.h>

// Merge two sorted sub-arrays arr[l..m] and arr[m+1..r]
void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int left[n1], right[n2];

    for (int i = 0; i < n1; i++) left[i] = arr[l + i];
    for (int j = 0; j < n2; j++) right[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    // Merge the two arrays back into arr in sorted order
    while (i < n1 && j < n2)
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];

    while (i < n1) arr[k++] = left[i++];
    while (j < n2) arr[k++] = right[j++];
}

// Recursively divide the array and sort each half
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
```

```
        merge(arr, l, m, r);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[100], n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    printArray(arr, n);

    mergeSort(arr, 0, n - 1);

    printf("Sorted array:   ");
    printArray(arr, n);

    return 0;
}
```

4. Output



Experiment 6: Quick Sort

Write a program to implement the quick sort using array.

1. Theory

Quick sort is a divide-and-conquer algorithm that chooses a pivot element and partitions the array so that smaller elements lie to its left and larger elements to its right. It then recursively sorts the two partitions. Its average time complexity is O(n log n), although a poor pivot choice can lead to a worst case of O(n^2). It is an in-place algorithm and is widely used due to its strong average-case performance.

2. Algorithm

1. Choose an element as the pivot (commonly the last element).
2. Partition the array so smaller elements lie left of the pivot and larger ones lie right.
3. Place the pivot at its correct sorted position.
4. Recursively apply the same process to the left sub-array.
5. Recursively apply the same process to the right sub-array.
6. Stop when each sub-array has one or zero elements.

3. Code

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Places pivot at correct position and partitions the array around it
int partition(int arr[], int low, int high) {
    int pivot = arr[high];   // choose last element as pivot
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Recursively sorts elements before and after partition
void quickSort(int arr[], int low, int high) {
```

```
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[100], n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    printArray(arr, n);
    quickSort(arr, 0, n - 1);
    printf("Sorted array:   ");
    printArray(arr, n);

    return 0;
}
```

4. Output

```
Enter the number of elements: 8
Enter 8 elements:
50 23 9 18 61 32 4 27
Original array: 50 23 9 18 61 32 4 27
Sorted array:   4 9 18 23 27 32 50 61
```

Experiment 7: Heap Sort

Write a program to implement the Heap Sort using array.

1. Theory

Heap sort uses a binary heap data structure to sort an array. It first builds a max heap from the input, placing the largest element at the root. It then repeatedly swaps the root with the last element of the heap, reduces the heap size, and restores the heap property. Its time complexity is O(n log n) in all cases. It is an in-place algorithm, though it is not stable.

2. Algorithm

1. Build a max heap from the input array.
2. Swap the root (maximum element) with the last element of the heap.
3. Reduce the heap size by one.
4. Heapify the root to restore the max heap property.
5. Repeat steps 2-4 until the heap size becomes 1.
6. Stop; the array is now sorted.

3. Code

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Maintains max-heap property for subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);   // re-heapify affected sub-tree
    }
}

// Builds max heap, then repeatedly extracts the maximum element
void heapSort(int arr[], int n) {
```

```
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);    // move current root to end
        heapify(arr, i, 0);        // heapify reduced heap
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[100], n;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Original array: ");
    printArray(arr, n);
    heapSort(arr, n);
    printf("Sorted array:   ");
    printArray(arr, n);

    return 0;
}
```

4. Output